SOFTWARE ENGINEERING · CHAPTER 06

Lab-06: Testing in Java Web Applications (JUnit, Mockito)

You receive a small, working Spring Boot 4.1 shop application, `shop-tests`, that has **no tests at all**. Across five tasks you build its complete test suite: plain JUnit 6 unit tests for the money and pricing rules, parameterized and repeated tests, Mockito-isolated service tests, Spring slice tests for the controller, repository and JWT filter, and finally integration tests on a random port and on PostgreSQL in Docker, guarded by a JaCoCo coverage gate and a GitHub Actions workflow. The challenge adds regression tests for real bug reports, a consumer-driven contract, a Playwright end-to-end test and mutation testing.

JUnit 6.1.3 · Mockito 5.23.0 · AssertJ | Spring Boot 4.1.1 test support | JDK 25 · Maven 3.9 · Docker (Task 5) | ≈ 5 hours | 5 tasks + 1 challenge

> **How to work through this lab.** Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes. Commit after every task; the grader reads your history.

0 Setup: the shop-tests project — 20 min

1. Verify the toolchain. Docker is needed for Task 5 only; the lab explains a fallback if you cannot install it.

```
java -version        # 25
mvn -v               # 3.9.x, or use ./mvnw inside the project
git --version
docker version       # optional until Task 5 (Docker Desktop, Rancher
Desktop or Podman)
```

2. Get the starter project from the course repository and build it once. It compiles and runs, but `src/test` is empty.

```
git clone <course-git>/shop-tests.git && cd shop-tests
./mvnw -q verify                # BUILD SUCCESS, "Tests run: 0"
```

```
./mvnw spring-boot:run           # http://localhost:8080/api/products needs
a JWT (see README)
```

3. Read the code you are going to test. Package `edu.itc.shop`; every class is small and has Javadoc describing its rules.

```
edu.itc.shop
├── ShopApplication.java
├── domain/      Money (record: amount, currency; of(String), plus,
times(int), percentOff(Percent), ZERO)
│                Percent (record, 0..100), Product (JPA entity: id, sku
unique, name, priceCents, stock),
│                OrderStatus (enum: NEW, PAID, SHIPPED, CANCELLED;
isEditable())
├── service/     PricingService(TaxTable): gross(net, country),
discounted(net, off),
│                  breakdown(net, country) -> List<PriceLine>   rules: DE
19, FR 20, IE 23, "" 0, else throws
│                IdGenerator: nextOrderId() -> "ORD-" + 8 base-36 upper-case
chars from SecureRandom
│                PriceClient(RestClient): currentPrice(sku) GET
/prices/{sku}; 5xx -> PriceUnavailableException
│                ProductService(ProductRepository, PriceClient, Clock):
create, get, search, restock
│                ProductNotFoundException, InsufficientStockException
├── repository/  ProductRepository extends JpaRepository<Product, Long>
│                  findByNameContainingIgnoreCase,
countByPriceCentsLessThan, findBySku
├── web/         ProductController (/api/products), ApiExceptionHandler
(ProblemDetail), dto/ ProductDto, CreateProduct
├── security/    SecurityConfig (stateless, /api/** needs a JWT, POST/PATCH
need ROLE_ADMIN),
│                JwtFilter (OncePerRequestFilter), JwtService (issue, parse
-> JwtClaims, throws JwtException)
└── util/        MoneyFormatter(Locale): format(Money) -> "49,90 €"
```

4. Confirm the test dependencies in `pom.xml`. `spring-boot-starter-test` already brings JUnit Jupiter 6.1, Mockito 5.23, AssertJ, Hamcrest, JSONassert, Awaitility and Spring Test. The others are used from Task 4 on.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
```

```
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-testcontainers</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.testcontainers</groupId>
        <artifactId>testcontainers-postgresql</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.testcontainers</groupId>
        <artifactId>testcontainers-junit-jupiter</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>test</scope>
    </dependency>
```

5. Create the test package tree, mirroring `src/main`. Tests for a class live in the same package name under `src/test/java`.

```
src/test/java/edu/itc/shop/
├── domain/       MoneyTest, PricingServiceTest (Task 1), ...ParamTest (Task
2)
├── service/      ProductServiceTest (Task 3), PriceClientTest (Task 5)
├── web/          ProductControllerTest (Task 4), ProductApiIT (Task 5)
├── repository/   ProductRepositoryTest (Task 4),
ProductRepositoryPostgresIT (Task 5)
├── security/     JwtFilterTest (Task 4)
└── testing/      ProductBuilder, Products (object mother, Task 3)
src/test/resources/
├── application-test.yml        H2 in-memory, fixed JWT secret, price API
base URL
└── fixtures/                   tax-rates.csv (Task 2), products.sql (Task
4)
```

6. Create `src/test/resources/application-test.yml` now; every Spring test in this lab activates the `test` profile.

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1
```

```
    username: sa
    password: ""
  jpa:
    hibernate:
      ddl-auto: create-drop
shop:
  jwt:
    secret: test-secret-test-secret-test-secret-1234   # 32+ bytes, tests
only
  price-api:
    base-url: http://localhost:9999                    # replaced by the
mock server
```

⚠ Do not change production code in `src/main` during Tasks 1 to 5 unless a task tells you to. If a test reveals a genuine bug, write it down in `NOTES.md`; the challenge is where you fix bugs.

1 Unit tests with JUnit 6: `Money` and `PricingService` Easy

40 min

GOAL

Write readable, well-structured JUnit Jupiter tests for two pure classes: lifecycle methods, `@Nested` contexts, `@DisplayName`, grouped assertions with `assertAll`, exception assertions and AssertJ's fluent collection assertions. No Spring, no Mockito yet.

STEPS

1. Create `domain/MoneyTest` with `@DisplayName("Money")`. Cover: `Money.of("10.00")` keeps scale 2; `plus` of two amounts; `times(3)`; `Money.of("-1.00")` throws `IllegalArgumentException` whose message contains `"negative"`; adding two different currencies throws.
2. Add a `@Nested @DisplayName("rounding")` class with tests that `Money.of("10.005")` rounds *half-up* to `10.01` and that `percentOff(Percent.of(15))` on `100.00` gives `85.00`.
3. Create `service/PricingServiceTest`. Load the `TaxTable` once in `@BeforeAll` (static field) and create a fresh `PricingService` in `@BeforeEach`. Add a `@AfterEach` that nulls the service; print nothing.
4. Test `gross(Money.of("10.00"), "DE")` with one `assertAll("gross")` containing three checks: amount equals `11.90`, currency is EUR, and `breakdown(...)` has 3 lines.
5. Test the unknown country `"XX"` with `assertThrows` and check the message contains `XX`. Put this in a `@Nested` class named `WhenCountryIsUnknown`.

6. Use AssertJ on `breakdown(Money.of("100.00"), "FR")`: `assertThat(lines).extracting(PriceLine::label, PriceLine::amount).containsExactly(tuple("net", ...), tuple("VAT 20 %", ...), tuple("gross", ...))`.
7. Run `./mvnw -q test`, then open the Surefire XML in `target/surefire-reports/` and confirm the display names appear. Commit: `task1: unit tests for Money and PricingService`.

STARTER CODE

```
package edu.itc.shop.domain;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.assertj.core.api.Assertions.assertThat;

@DisplayName("Money")
class MoneyTest {

    @Test
    @DisplayName("of() keeps two decimals")
    void ofKeepsScale() {
        Money m = Money.of("10");
        assertEquals(2, m.amount().scale());
        assertEquals(new BigDecimal("10.00"), m.amount());
    }

    @Test
    void negativeAmountIsRejected() {
        // TODO assertThrows + message check
    }

    @Nested
    @DisplayName("rounding")
    class Rounding {
        // TODO half-up rounding, percentOff
    }
}
```

EXPECTED OUTPUT

```
$ ./mvnw test
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.itc.shop.domain.MoneyTest
```

```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in
edu.itc.shop.domain.MoneyTest
[INFO] Running edu.itc.shop.service.PricingServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in
edu.itc.shop.service.PricingServiceTest
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# make one assertion wrong on purpose to see a grouped failure, then revert:
org.opentest4j.MultipleFailuresError: gross (1 failure)
    org.opentest4j.AssertionFailedError: expected: <Money[amount=11.90,
currency=EUR]>
                                         but was: <Money[amount=11.19,
currency=EUR]>
```

Show hints

- `@BeforeAll` and `@AfterAll` must be `static` unless you add `@TestInstance(Lifecycle.PER_CLASS)`. Keep the default and make the field static.
- Compare `BigDecimal` with `assertEquals(new BigDecimal("11.90"), ...)` only when the scale matches; AssertJ's `isEqualByComparingTo("11.90")` ignores scale.
- `assertThrows` returns the exception: `var ex = assertThrows(IllegalArgumentException.class, () -> Money.of("-1.00"));`
- `tuple(...)` is `org.assertj.core.groups.Tuple`, statically imported from `org.assertj.core.api.Assertions`.
- A `@Nested` class is a non-static inner class; it can use the outer class's fields set in the outer `@BeforeEach`.

ACCEPTANCE CHECKLIST

- [ ] `MoneyTest` has at least 7 tests including a `@Nested` rounding class; all pass.
- [ ] `PricingServiceTest` uses `@BeforeAll` (static), `@BeforeEach`, `@AfterEach` and a `@Nested` unknown-country context.
- [ ] One `assertAll` with three checks and one `assertThrows` that inspects the message.
- [ ] AssertJ `extracting(...).containsExactly(...)` is used on the breakdown; every test has a meaningful name or `@DisplayName`.

2 Parameterized and repeated tests Easy 40 min

GOAL

Turn the tax and discount rules into data-driven tests with `@CsvSource`, `@CsvFileSource`, `@MethodSource` and `@EnumSource`, cover the edge cases explicitly, and probe the random-based `IdGenerator` with `@RepeatedTest`.

STEPS

1. Create `service/PricingRulesParamTest`. Add a `@ParameterizedTest(name = "[{index}] {0} → {1} %")` with `@CsvSource` for the VAT rates `DE 19, FR 20, IE 23` and the quoted empty country `'' → 0`.
2. Move the same table into `src/test/resources/fixtures/tax-rates.csv` (with a header line) and add a second test using `@CsvFileSource(resources = "/fixtures/tax-rates.csv", numLinesToSkip = 1)` that checks `gross(net, country)` for a `net` column too.
3. Add `@MethodSource("discountCases")` feeding `Money net, Percent off, Money expected`. Cases: 0 %, 15 % on 100.00, 50 % on 0.01 (stays 0.01), 100 % (gives 0.00), and 33 % on 10.00 (6.70, half-up).
4. Add `@NullAndEmptySource` + `@ValueSource(strings = {" ", "\t"})` on a test asserting that `new Product(sku, "Name", 100)` rejects a blank SKU.
5. Add `@EnumSource(value = OrderStatus.class, names = {"PAID", "SHIPPED", "CANCELLED"})` asserting `isEditable()` is false, and a second test with `mode = EXCLUDE` asserting the rest are editable.
6. Create `service/IdGeneratorTest` with `@RepeatedTest(value = 200, name = "id {currentRepetition}/{totalRepetitions}")`: the id matches `ORD-[0-9A-Z]{8}` and is added to a static `Set` without collision. Inject `RepetitionInfo` into the failure message.
7. Run `./mvnw -q test -Dtest='*ParamTest,IdGeneratorTest'`; look at the per-invocation names in your IDE's test tree. Commit: `task2: parameterized and repeated tests`.

STARTER CODE

```
package edu.itc.shop.service;

import java.util.stream.Stream;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.params.provider.Arguments.arguments;
import static org.junit.jupiter.api.Assertions.*;

class PricingRulesParamTest {

    static final TaxTable taxes = TaxTable.load("eu-2026");
    final PricingService pricing = new PricingService(taxes);
```

```
    @ParameterizedTest(name = "[{index}] {0} → {1} %")
    @CsvSource({
        "DE, 19",
        // TODO FR, IE and the quoted empty country
    })
    void vatRatePerCountry(String country, int expectedRate) {
        assertEquals(expectedRate, taxes.rateFor(country));
    }

    @ParameterizedTest
    @MethodSource("discountCases")
    void discountIsApplied(Money net, Percent off, Money expected) {
        assertEquals(expected, pricing.discounted(net, off));
    }

    static Stream<Arguments> discountCases() {
        return Stream.of(
            arguments(Money.of("100.00"), Percent.of(0), Money.of("100.00"))
            // TODO four more cases including the edge cases
        );
    }
}
```

```
net,country,gross
10.00,DE,11.90
100.00,FR,120.00
9.99,IE,12.29
10.00,,10.00
```

EXPECTED OUTPUT

```
PricingRulesParamTest
  vatRatePerCountry(String, int)
    ✔ [1] DE → 19 %
    ✔ [2] FR → 20 %
    ✔ [3] IE → 23 %
    ✔ [4]  → 0 %
  grossFromCsvFile(String, String, String)      4 invocations ✔
  discountIsApplied(Money, Percent, Money)      5 invocations ✔
  blankSkuIsRejected(String)                    4 invocations ✔   (null, "", "
", "\t")
  closedStatusesAreNotEditable(OrderStatus)     3 invocations ✔
  openStatusesAreEditable(OrderStatus)          1 invocation  ✔   (NEW)
IdGeneratorTest
  id 1/200 ... id 200/200                       200 invocations ✔
```

```
[INFO] Tests run: 232, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- In `@CsvSource` an unquoted empty cell becomes `null`; the quoted `''` becomes the empty string. Your test needs the empty string.
- `@CsvFileSource` reads from the classpath: the file must be under `src/test/resources`. The empty country cell in the file is `null` unless you set `emptyValue` or `nullValues`; handle it in the test with `Objects.requireNonNullElse(country, "")`.
- Argument conversion: JUnit converts `"19"` to `int` automatically. For `Money` in a CSV column read a `String` and call `Money.of` in the test body, or add a static factory method named `of(String)` or `valueOf(String)`, which JUnit also finds.
- The `Set` shared by the 200 repetitions must be `static`, because JUnit creates a new test instance per repetition. Use `ConcurrentHashMap.newKeySet()`.
- Filter parameterized tests from the command line: `-Dtest='PricingRulesParamTest#vatRatePerCountry'`.

ACCEPTANCE CHECKLIST

- [ ] VAT rules are tested from `@CsvSource` and from `fixtures/tax-rates.csv`, including the empty country.
- [ ] `@MethodSource` covers 0 %, 100 %, the 0.01 floor and a half-up rounding case.
- [ ] Blank SKU edge cases use `@NullAndEmptySource`; `@EnumSource` is used with `names` and with `mode = EXCLUDE`.
- [ ] `IdGeneratorTest` runs 200 repetitions, checks the format and uniqueness, and uses `RepetitionInfo` in its message.

3 Mockito: isolating `ProductService` Medium 60 min

GOAL

Test `ProductService` as a unit: its repository and price client are Mockito mocks, time is a fixed `Clock`. Practise stubbing, `ArgumentCaptor`, ordered and negative verification, a spy on the real `MoneyFormatter`, and experience the strict-stubs failure on purpose. Introduce a test data builder so the tests stay short.

STEPS

1. Create `testing/ProductBuilder` (`aProduct().withSku(..).withName(..).withPriceCents(..).withStock(..).build()`, unique default SKU) and `testing/Products` (object mother: `keyboard()`, `soldOutMouse()`).
2. Create `service/ProductServiceTest` with `@ExtendWith(MockitoExtension.class)`, `@Mock ProductRepository repo`, `@Mock PriceClient prices`, a `Clock.fixed(...)` and the service built in `@BeforeEach`.
3. **create**: stub `prices.currentPrice("KB-01")` to `Money.of("49.90")` and `repo.save(any())` with `thenAnswer` that returns its argument. Capture the saved `Product` with an `ArgumentCaptor` and assert sku, priceCents 4990 and `createdAt == clock.instant()`. Use `InOrder` to prove the price is fetched *before* the save.
4. **create when the price API is down**: stub `currentPrice` with `thenThrow(new PriceUnavailableException("503"))`; assert the exception propagates and `verify(repo, never()).save(any())`.
5. **get / restock**: `findById(42L)` empty → `ProductNotFoundException` with message `"Product 42 not found"`; `restock(id, -10)` on stock 5 → `InsufficientStockException`, `verifyNoInteractions(prices)`.
6. Add `@Spy MoneyFormatter formatter = new MoneyFormatter(Locale.GERMANY)` in a small `MoneyFormatterSpyTest`: the real call returns `"49,90 €"`; then `doReturn("n/a").when(formatter).format(Money.ZERO)`; verify `times(2)`.
7. **Strict stubs demo**: add a test with a stub that is never used, run it, copy the `UnnecessaryStubbingException` message into `NOTES.md`, then fix the test (remove the stub, or justify `lenient()` in a comment). Commit: `task3: ProductService unit tests with Mockito`.

STARTER CODE

```
package edu.itc.shop.service;

import java.time.*;
import java.util.Optional;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import static edu.itc.shop.testing.ProductBuilder.aProduct;
import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock ProductRepository repo;
    @Mock PriceClient prices;
```

```
    @Captor ArgumentCaptor<Product> savedProduct;
    final Clock clock = Clock.fixed(Instant.parse("2026-09-25T10:00:00Z"),
ZoneOffset.UTC);
    ProductService service;

    @BeforeEach
    void setUp() { service = new ProductService(repo, prices, clock); }

    @Test
    void createFetchesThePriceThenSavesTheProduct() {
        when(prices.currentPrice("KB-01")).thenReturn(Money.of("49.90"));
        when(repo.save(any(Product.class))).thenAnswer(inv ->
inv.getArgument(0));

        ProductDto dto = service.create(new CreateProduct("KB-01", "Mechanical
Keyboard"));

        // TODO captor: sku, priceCents, createdAt
        // TODO InOrder: prices.currentPrice(..) before repo.save(..)
        assertThat(dto.priceCents()).isEqualTo(4990);
    }

    @Test
    void createDoesNotSaveWhenThePriceApiIsDown() {
        // TODO thenThrow, assertThatThrownBy, verify(repo, never()).save(any())
    }
}
```

EXPECTED OUTPUT

```
# step 7, before the fix:
[ERROR] ProductServiceTest.getReturnsExistingProduct
org.mockito.exceptions.misusing.UnnecessaryStubbingException:
Unnecessary stubbings detected.
Clean & maintainable test code requires zero unnecessary code.
Following stubbings are unnecessary (click to navigate to relevant line of
code):
  1. -> at
edu.itc.shop.service.ProductServiceTest.getReturnsExistingProduct(ProductService
Test.java:61)
Please remove unnecessary stubbings or use 'lenient' strictness. More info:
javadoc for UnnecessaryStubbingException class.

# after the fix:
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in
edu.itc.shop.service.ProductServiceTest
```

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in
edu.itc.shop.util.MoneyFormatterSpyTest
```

Show hints

- `verify(repo).save(savedProduct.capture());` then `savedProduct.getValue()`. With `@Captor` the extension creates the captor for you.
- `InOrder inOrder = inOrder(prices, repo); inOrder.verify(prices).currentPrice("KB-01"); inOrder.verify(repo).save(any());`
- Do not mock `Money`, `Product` or `Clock`: they are values or have a perfectly good fake (`Clock.fixed`). Mocking them is a deduction.
- Stubbing a spy with `when(spy.format(..))` calls the real method once; use `doReturn(..).when(spy).format(..)` instead.
- If Mockito prints a warning about dynamically loading an agent on JDK 25, add `<argLine>@{argLine} -XX:+EnableDynamicAgentLoading</argLine>` to Surefire (the JaCoCo step in Task 5 needs the `@{argLine}` part).

ACCEPTANCE CHECKLIST

- [ ] `ProductServiceTest` uses `MockitoExtension`, `@Mock`, `thenReturn`, `thenAnswer` and `thenThrow`; the class under test and value objects are real.
- [ ] An `ArgumentCaptor` checks the saved product; `InOrder` proves price-before-save; `never()` and `verifyNoInteractions` are used.
- [ ] A `@Spy` on `MoneyFormatter` shows real behavior and a `doReturn` partial stub.
- [ ] `NOTES.md` contains the `UnnecessaryStubbingException` excerpt and the commit history shows the fix; `ProductBuilder` and `Products` exist and are used.

4 Slice tests: `@WebMvcTest`, `@DataJpaTest` and the JWT filter

Medium **60 min**

GOAL

Test each Spring layer in its own slice: the controller through `MockMvcTester` with a `@MockitoBean` service, the repository's derived queries on an embedded database with `TestEntityManager` and an `@Sql` fixture, and the `JwtFilter` inside the real security chain with MockMvc.

STEPS

1. Create `web/ProductControllerTest` with `@WebMvcTest(ProductController.class)`, `@Autowired MockMvcTester mvc` and `@MockitoBean ProductService service`. Tests: `GET /api/products` with `@WithMockUser` returns 200 and a JSON array whose first `name` is `"Mechanical Keyboard"`; `GET /api/products/999` returns 404 `application/problem+json` when the service throws `ProductNotFoundException`.
2. Add: `POST` with `{"sku":"KB-01","name":""}` as `@WithMockUser(roles = "ADMIN")` returns 400 with `$.errors[0].field == "name"` and the service is never called; the same POST as a plain user returns 403; a request without any user returns 401.
3. Create `repository/ProductRepositoryTest` with `@DataJpaTest` and `@ActiveProfiles("test")`. With `TestEntityManager` persist two products, `flush()`, `clear()`, then assert `findByNameContainingIgnoreCase("KEYBOARD")` returns exactly the keyboard's SKU and `findBySku("nope")` is empty.
4. Add a test that persisting two products with the same SKU throws a `PersistenceException` on `persistAndFlush` (unique constraint).
5. Create `src/test/resources/fixtures/products.sql` with three rows (see the chapter) and a test class `ProductQueryTest` annotated `@Sql("/fixtures/products.sql")` asserting `countByPriceCentsLessThan(2000) == 2`.
6. Create `security/JwtFilterTest`: `@WebMvcTest(ProductController.class)` + `@Import({SecurityConfig.class, JwtFilter.class})`, `@MockitoBean ProductService`, `@MockitoBean JwtService jwt`. Tests: no header → 401 and `jwt` never called; `Bearer good` with `jwt.parse` stubbed to `new JwtClaims("alice", List.of("ROLE_USER"))` → 200 and `authenticated().withUsername("alice")`; `Bearer old` with `willThrow(new JwtException("expired"))` → 401 and `$.detail == "expired"`.
7. Run `./mvnw -q test`; note in `NOTES.md` how long the slice tests take compared with Task 1. Commit: `task4: slice tests for web, JPA and security`.

STARTER CODE

```
package edu.itc.shop.web;

import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.assertj.MockMvcTester;
import org.springframework.security.test.context.support.WithMockUser;
import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.BDDMockito.*;

@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvcTester mvc;
    @MockitoBean ProductService service;
```

```
    @Test
    @WithMockUser
    void listReturnsProducts() {
        given(service.search(null)).willReturn(List.of(
            new ProductDto(1L, "KB-01", "Mechanical Keyboard", 4990, 5)));

        assertThat(mvc.get().uri("/api/products"))
            .hasStatusOk()
            .hasContentType(MediaType.APPLICATION_JSON)
            .bodyJson().extractingPath("$[0].name").isEqualTo("Mechanical
Keyboard");
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void createWithBlankNameIs400() {
        assertThat(mvc.post().uri("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {"sku": "KB-01", "name": ""}
                    """))
            .hasStatus(HttpStatus.BAD_REQUEST)
            .hasContentType(MediaType.APPLICATION_PROBLEM_JSON)
            .bodyJson().extractingPath("$.errors[0].field").isEqualTo("name");
        then(service).shouldHaveNoInteractions();
    }
    // TODO 404, 403, 401
}


@DataJpaTest
@ActiveProfiles("test")
class ProductRepositoryTest {
    @Autowired TestEntityManager em;
    @Autowired ProductRepository repo;

    @Test
    void findsByNameIgnoringCase() {
        em.persist(aProduct().withSku("KB-01").withName("Mechanical
Keyboard").build());
        em.persist(aProduct().withSku("MS-01").withName("Mouse").build());
        em.flush();
        em.clear();
        // TODO assert on findByNameContainingIgnoreCase("KEYBOARD")
    }
}
```

EXPECTED OUTPUT

```
[INFO] Running edu.itc.shop.web.ProductControllerTest
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \        (the banner appears once per
test context)
 :: Spring Boot ::                (v4.1.1)
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.9 s --
in ProductControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.4 s --
in ProductRepositoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.3 s --
in ProductQueryTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.6 s --
in JwtFilterTest

# what a failing MockMvcTester assertion looks like (for orientation):
java.lang.AssertionError: [Status] expected: 200 OK but was: 401 UNAUTHORIZED
MvcTestResult: GET /api/products ... Headers = [WWW-Authenticate:"Bearer", ...]
```

Show hints

- Spring Boot 4: `@WebMvcTest` is in `org.springframework.boot.webmvc.test.autoconfigure`, `@DataJpaTest` and `TestEntityManager` in `org.springframework.boot.data.jpa.test.autoconfigure`. `@MockitoBean` comes from Spring Framework (`org.springframework.test.context.bean.override.mockito`).
- Without importing `SecurityConfig`, the slice uses Boot's default security (any authenticated user, CSRF on). The starter's `SecurityConfig` disables CSRF for the stateless API; import it in `ProductControllerTest` too if your POST gets 403 with `@WithMockUser(roles = "ADMIN")`.
- `authenticated().withUsername("alice")` is `SecurityMockMvcResultMatchers.authenticated()` from `spring-security-test`; it works with classic `MockMvc`, so use `@Autowired MockMvc mvc` in `JwtFilterTest`.
- `em.clear()` detaches everything so the repository call hits the database instead of the first-level cache; without it a wrong query can still "pass".
- The 401 for an expired token is produced by the filter writing a `ProblemDetail` or by an `AuthenticationEntryPoint`; read `JwtFilter` to see which and assert accordingly.

ACCEPTANCE CHECKLIST

- [ ] `ProductControllerTest` covers 200, 404 (problem+json), 400 with field error, 403 and 401 using `MockMvcTester` and a `@MockitoBean`.
- [ ] `@DataJpaTest` tests use `TestEntityManager` with `flush`/`clear`, cover the derived query, `findBySku` and the unique constraint.

- [ ] `@Sql` fixture `products.sql` drives `ProductQueryTest`; no test depends on another test's data.
- [ ] `JwtFilterTest` proves 401 without token, 200 + authenticated principal with a valid token, 401 with detail for an expired one.

5 Integration tests, coverage gate and CI

Hard — **90 min**

GOAL

Complete the pyramid: a full-stack test on a random port with `TestRestTemplate`, the repository on real PostgreSQL through Testcontainers and `@ServiceConnection` (with an H2 fallback when Docker is absent), `MockRestServiceServer` for the external price API, a JaCoCo 80 % line-coverage gate, and a GitHub Actions workflow that runs `./mvnw verify` on every push.

STEPS

1. Configure `maven-failsafe-plugin` in `pom.xml` (goals `integration-test` and `verify`) so that classes named `*IT` run in the `verify` phase and are excluded from Surefire.
2. Create `web/ProductApiIT`: `@SpringBootTest(webEnvironment = RANDOM_PORT)`, `@ActiveProfiles("test")`, `@Autowired TestRestTemplate rest`, `@Autowired JwtService jwt`. Issue an admin token, `POST /api/products` → 201 with a `Location` header, then `GET` the location → 200 and the same name. Second test: `GET /api/products/999` → 404 whose `detail` is `"Product 999 not found"`. Note: this IT runs on H2 (profile `test`), but `PriceClient` needs a stub: declare a `@MockitoBean PriceClient` returning `Money.of("49.90")`.
3. Create `repository/ProductRepositoryPostgresIT`: `@DataJpaTest` + `@Testcontainers(disabledWithoutDocker = true)` + a `static @Container @ServiceConnection PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine")`. Repeat the derived-query test and add one that only PostgreSQL can answer: `findByNameContainingIgnoreCase("ß")` or a query using `ILIKE` from the starter's `@Query` method `searchByPattern`. Without Docker the class is reported as *skipped*, not failed: that is the fallback.
4. Create `service/PriceClientTest` with `@RestClientTest(PriceClient.class)`, `@Autowired MockRestServiceServer server`: a 200 JSON body parses to `Money.of("49.90")` and `server.verify()` passes; a `withServerError()` response becomes `PriceUnavailableException`; a request to the wrong path fails the test (try it once, then remove).

5. Add `jacoco-maven-plugin` 0.8.14 with `prepare-agent`, `report` (phase `verify`) and `check` with a `BUNDLE` rule `LINE COVEREDRATIO ≥ 0.80`; exclude `ShopApplication` and `**/dto/*`. Run `./mvnw verify`, open `target/site/jacoco/index.html`, and write the three least-covered classes into `NOTES.md`. If the gate fails, add tests (not exclusions) until it passes.
6. Create `.github/workflows/ci.yml`: trigger on push and pull request, `ubuntu-latest`, `actions/checkout@v5`, `actions/setup-java@v5` (temurin 25, `cache: maven`), `./mvnw -B verify`, and upload `target/surefire-reports`, `target/failsafe-reports` and `target/site/jacoco` as artifacts with `if: always()`. Push to GitHub (or the course GitLab with the equivalent `.gitlab-ci.yml`) and link the green run in `README.md`.
7. Commit: `task5: integration tests, JaCoCo gate, CI workflow`. Verify from a clean clone that `./mvnw verify` is green with Docker running and also green (with the Postgres IT skipped) without Docker.

STARTER CODE

```
package edu.itc.shop.repository;

import
org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

@DataJpaTest
@Testcontainers(disabledWithoutDocker = true)   // H2 fallback: this class is
skipped, not failed
class ProductRepositoryPostgresIT {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-
alpine");

    @Autowired ProductRepository repo;

    @Test
    void derivedQueryRunsOnRealPostgres() {
        repo.save(Products.keyboard());
        // TODO assert findByNameContainingIgnoreCase("KEYBOARD") -> ["KB-01"]
    }
}


@RestClientTest(PriceClient.class)
@TestPropertySource(properties = "shop.price-api.base-url=")   // relative URLs
```

```
hit the mock server
class PriceClientTest {
    @Autowired PriceClient client;
    @Autowired MockRestServiceServer server;

    @Test
    void parsesThePrice() {
        server.expect(requestTo("/prices/KB-01"))
              .andExpect(method(HttpMethod.GET))
              .andRespond(withSuccess("""
                  {"sku":"KB-01","amount":"49.90","currency":"EUR"}
                  """, MediaType.APPLICATION_JSON));

        assertThat(client.currentPrice("KB-01")).isEqualTo(Money.of("49.90"));
        server.verify();
    }
    // TODO withServerError() -> PriceUnavailableException
}


<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-failsafe-plugin</artifactId>
    <executions>
        <execution>
            <goals><goal>integration-test</goal><goal>verify</goal></goals>
        </execution>
    </executions>
</plugin>
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.14</version>
    <executions>
        <execution><goals><goal>prepare-agent</goal></goals></execution>
        <execution><id>report</id><phase>verify</phase><goals>
<goal>report</goal></goals></execution>
        <execution>
            <id>coverage-gate</id><phase>verify</phase><goals><goal>check</goal>
</goals>
            <configuration>
                <rules><rule><element>BUNDLE</element><limits><limit>
                    <counter>LINE</counter><value>COVEREDRATIO</value>
<minimum>0.80</minimum>
                </limit></limits></rule></rules>
                <excludes>
                    <exclude>**/ShopApplication.class</exclude>
                    <exclude>**/dto/*</exclude>
```

```
                </excludes>
            </configuration>
        </execution>
    </executions>
</plugin>


# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main, 'lab-*']
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: '25', cache: maven }
      - run: ./mvnw -B --no-transfer-progress verify
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: reports
          path: |
            target/surefire-reports
            target/failsafe-reports
            target/site/jacoco
```

EXPECTED OUTPUT

```
$ ./mvnw verify
[INFO] --- surefire:3.5.x:test (default-test) @ shop-tests ---
[INFO] Tests run: 262, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- failsafe:3.5.x:integration-test (default) @ shop-tests ---
[INFO] Running edu.itc.shop.repository.ProductRepositoryPostgresIT
... tc.postgres:17-alpine: Container postgres:17-alpine started in PT2.8S
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in
ProductRepositoryPostgresIT
[INFO] Running edu.itc.shop.web.ProductApiIT
[INFO] Tomcat started on port 51733 (http) with context path '/'
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in ProductApiIT
[INFO] --- jacoco:0.8.14:report (report) @ shop-tests ---
[INFO] Analyzed bundle 'shop-tests' with 23 classes
[INFO] --- jacoco:0.8.14:check (coverage-gate) @ shop-tests ---
```

```
[INFO] All coverage checks have been met.
[INFO] BUILD SUCCESS

# without Docker:
[WARNING] Tests run: 2, Failures: 0, Errors: 0, Skipped: 2 -- in
ProductRepositoryPostgresIT
   (disabledWithoutDocker: Docker is not available)
```

Show hints

- Since Spring Boot 3.4 a `@DataJpaTest` keeps a `@ServiceConnection` DataSource instead of replacing it with H2; you do not need `@AutoConfigureTestDatabase(replace = NONE)`.
- The container image is pulled on first use (about 100 MB). In CI on `ubuntu-latest` Docker is already running; nothing to install.
- `TestRestTemplate` does not throw on 4xx/5xx, so `rest.getForEntity("/api/products/999", ProblemDetail.class)` gives you the status and the body in one call. Add the bearer token with an `HttpEntity` and `rest.exchange(...)`, or `rest.getRestTemplate().getInterceptors().add(...)` once in `@BeforeEach`.
- JaCoCo's `prepare-agent` writes the `argLine` property; if your Surefire configuration sets `<argLine>`, it must start with `@{argLine}` or coverage will be 0 %.
- If the gate fails, the log names the counter: `Rule violated for bundle shop-tests: lines covered ratio is 0.74, but expected minimum is 0.80`. Sort the JaCoCo report by "Missed Lines" to find the cheapest wins.

ACCEPTANCE CHECKLIST

- [ ] `ProductApiIT` runs on a random port with `TestRestTemplate`, covers the 201 + Location + GET round trip and a 404 problem detail; `*IT` classes run under Failsafe.
- [ ] `ProductRepositoryPostgresIT` uses `@Testcontainers`, `@Container`, `@ServiceConnection`, no datasource properties, and is skipped (not failed) without Docker.
- [ ] `PriceClientTest` uses `@RestClientTest` and `MockRestServiceServer` with `server.verify()` and an error-mapping case.
- [ ] JaCoCo `check` enforces 80 % lines and passes; the GitHub Actions run is green and linked from `README.md`.

★

Challenge: regression suite, consumer contract, e2e test, mutation testing

Hard **120–150 min · optional, extra credit**

SCENARIO

The shop went live and three bug reports came in. The price API team asks for a contract so they can evolve their service safely. Product management wants proof that the product page works in a real browser. And your lead suspects that 80 % coverage hides weak assertions. Deliver all four, with a written analysis.

REQUIREMENTS

1. **Regression suite** in `src/test/java/edu/itc/shop/regression/`, one test per ticket, named `bug101_...` etc., each with a Javadoc quoting the report. Write the test first, watch it fail, then fix `src/main`:
   - **#101** "Restocking with a negative delta larger than the stock leaves the stock negative." (`ProductService.restock` checks `<` instead of `<=` on zero.)
   - **#102** "Searching `'keyboard '` with a trailing space returns nothing, although the product exists." (`search` does not trim.)
   - **#103** "A 33 % discount on 10.00 shows 6.69 in the cart but 6.70 on the invoice." (`PricingService.discounted` uses `RoundingMode.HALF_EVEN`, `Money` uses `HALF_UP`.)
2. **Consumer-driven contract** for `GET /prices/{sku}` with Pact JVM (`au.com.dius.pact.consumer:junit5`) or Spring Cloud Contract. The consumer test must run against the Pact mock server, produce `target/pacts/shop-price-api.json`, and only contain the fields `PriceClient` actually reads. Commit the pact file.
3. **End-to-end test** with Playwright for Java (`com.microsoft.playwright:playwright`) tagged `@Tag("e2e")`: start the app (`./mvnw spring-boot:run` or `@SpringBootTest(RANDOM_PORT)`), open `/products/KB-01`, assert the heading and the formatted price, click *Add to cart*, assert the cart counter. Select elements by role and `data-testid`. Exclude the tag from the default build and document how to run it.
4. **Mutation testing** with PIT (`org.pitest:pitest-maven` plus `pitest-junit5-plugin`) on `edu.itc.shop.domain` and `edu.itc.shop.service`. Write `MUTATION.md`: mutation score before and after, a table of at least five surviving mutants with the class, line, mutator, why the mutant survived (missing assertion? untested branch? equivalent mutant?) and what you changed. Kill every survivor that is not equivalent.

SKELETON

```
/**
 * Bug #101: "Restocking with a negative delta larger than the stock leaves the
stock negative."
 * Reported 2026-09-18 by support. Fixed in commit <sha>.
 */
@Test
void bug101_restockBelowZeroIsRejectedAndStockIsUnchanged() {
```

```
        Product p = aProduct().withStock(5).build();
        when(repo.findById(1L)).thenReturn(Optional.of(p));

        assertThatThrownBy(() -> service.restock(1L, -6))
            .isInstanceOf(InsufficientStockException.class);
        assertThat(p.stock()).isEqualTo(5);
        verify(repo, never()).save(any());
    }


    <plugin>
        <groupId>org.pitest</groupId>
        <artifactId>pitest-maven</artifactId>
        <version>1.22.0</version>   <!-- check the latest release; the JUnit 5
    plugin must support JUnit Platform 6 -->
        <dependencies>
            <dependency>
                <groupId>org.pitest</groupId>
                <artifactId>pitest-junit5-plugin</artifactId>
                <version>1.2.4</version>
            </dependency>
        </dependencies>
        <configuration>
            <targetClasses><param>edu.itc.shop.domain.*</param>
    <param>edu.itc.shop.service.*</param></targetClasses>
            <targetTests><param>edu.itc.shop.*Test</param></targetTests>
            <outputFormats><param>HTML</param></outputFormats>
        </configuration>
    </plugin>
```

EXPECTED OUTPUT

```
$ ./mvnw -q test -Dtest='edu.itc.shop.regression.*'      # before the fixes
[ERROR] Tests run: 3, Failures: 3 ...                    # after the fixes:
Failures: 0

$ ./mvnw -q test -Dtest=PriceClientPactTest && ls target/pacts
shop-price-api.json

$ ./mvnw -q test -Dgroups=e2e -Dshop.url=http://localhost:8080
[INFO] Tests run: 1, Failures: 0 -- in edu.itc.shop.e2e.ProductPageE2E

$ ./mvnw org.pitest:pitest-maven:mutationCoverage
================================================================================
- Statistics
================================================================================
>> Line Coverage (domain+service): 96%
```

```
>> Generated 118 mutations Killed 97 (82%)
>> Mutations with no coverage 4. Test strength 85%
# open target/pit-reports/index.html and analyse the survivors in MUTATION.md
```

Show hints

- Regression tests belong at the lowest level that reproduces the bug: #101 and #102 are service unit tests with mocks, #103 is a pure `PricingService` test. Add the cases to the parameterized sources of Task 2 as well.
- Pact: annotate the class with `@ExtendWith(PactConsumerTestExt.class)` and `@PactTestFor(providerName = "price-api")`; the `@Pact(consumer = "shop")` method returns a `V4Pact`; the test receives a `MockServer` whose `getUrl()` you pass to a `RestClient`. Use `decimalType` and `stringType` matchers, not literal values, so the provider can return any valid price.
- Playwright: run `./mvnw exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install chromium"` once. `assertThat(locator).hasText(..)` comes from `com.microsoft.playwright.assertions.PlaywrightAssertions` and auto-waits.
- Typical survivors: boundary mutants (`<` to `<=`) where no test hits the exact boundary; "removed call" mutants where a `verify` is missing; return-value mutants where the test only asserts *that* a result exists. An equivalent mutant cannot be killed by any test; say why.
- Keep PIT out of the default build: run it manually or in a separate CI job; a full run takes minutes.

ACCEPTANCE CHECKLIST

- [ ] Three regression tests, each committed red first and then green with the fix; Javadoc quotes the report.
- [ ] A consumer contract test passes against the Pact (or SCC) mock server and the generated contract file is committed and contains only the consumed fields.
- [ ] A Playwright e2e test drives the product page in a real browser, is tagged `e2e`, excluded from the default build and documented.
- [ ] `MUTATION.md` reports the PIT score before and after and analyses at least five survivors; non-equivalent survivors are killed by new assertions.

Grading rubric

Component	Points	Full marks when…
Task 1 · Unit tests with JUnit 6	10	lifecycle methods used correctly, `@Nested` contexts, `assertAll`, `assertThrows` with message check, AssertJ `extracting`; names

Component	Points	Full marks when…
		read as sentences
Task 2 · Parameterized and repeated	15	CSV inline and file, `@MethodSource` with objects, `@EnumSource` include/exclude, null/empty edge cases, 200 repetitions with uniqueness check
Task 3 · Mockito	20	only boundaries mocked, captor and `InOrder` used purposefully, negative verification, spy with `doReturn`, strict-stubs incident documented and fixed, builder and object mother in use
Task 4 · Slice tests	20	controller slice covers 200/400/401/403/404 with `@MockitoBean`; JPA slice uses `TestEntityManager` and `@Sql`; filter proven inside the security chain
Task 5 · Integration, coverage, CI	25	random-port IT, Testcontainers with `@ServiceConnection` and Docker-less skip, `MockRestServiceServer` with `verify()`, JaCoCo gate at 80 % passing, green CI run linked
Test code quality	10	AAA structure, no logic in tests, no duplication beyond builders, packages mirror main, `NOTES.md` is informative, commits per task
★ Challenge (extra credit)	+20	regression red-then-green history (5), contract with matchers (5), e2e by role/testid (4), mutation analysis with killed survivors (6)
Total	**100 (+20)**	

Automatic deductions: `Thread.sleep` anywhere in tests (-5) · tests that depend on execution order or shared mutable static state, including `@TestMethodOrder` to make them pass (-5) · a test method without any assertion or verification (-3 each, max -9) · mocking the class under test, `Money`, `Product` or `Clock` (-5) · production code changed in Tasks 1–5 without a note (-5) · coverage gate lowered below 0.80, disabled, or hidden by excluding tested classes (-10) · secrets or absolute local paths committed (-5).

Submission

1. Repository layout (the starter plus your additions):

```
shop-tests/
├── README.md                    how to run unit, integration and e2e tests;
```

```
link to the green CI run
├── NOTES.md                    strict-stubs excerpt, slice timings, least-
covered classes, observations
├── MUTATION.md                 (challenge) PIT analysis
├── .github/workflows/ci.yml
├── pom.xml                     surefire + failsafe + jacoco (+ pitest for
the challenge)
├── src/main/java/edu/itc/shop/  unchanged, except bug fixes in the
challenge
└── src/test/
    ├──
java/edu/itc/shop/{domain,service,web,repository,security,testing,regression
,e2e}/
    └── resources/{application-test.yml, fixtures/tax-rates.csv,
fixtures/products.sql}
```

2. From a clean clone, with Docker running, this must pass: `./mvnw -B verify` (all unit tests, both ITs, JaCoCo gate). Without Docker it must also pass, with `ProductRepositoryPostgresIT` reported as skipped.
3. Branch `lab-06`, merge request titled `Lab-06 – <your name>`. The MR description lists the number of tests per level (unit / slice / integration) and the JaCoCo line percentage.

Back to Chapter 06 slides | Retake the chapter quiz | ← Lab-05